

```
import Pkg; Pkg.add("PrettyTables")
using PrettyTables
using LinearAlgebra
```

Resolving package versions...
No Changes to `~/julia/environments/v1.11/Project.toml`
No Changes to `~/julia/environments/v1.11/Manifest.toml`

```
function simplexBigM(A, b, z, sense; mode=:max)
    # Get dimensions.
    m, n = size(A)

    # Preprocess: ensure b is nonnegative.
    for i in 1:m
        if b[i] < 0
            b[i] = -b[i]
            A[i, :] = -A[i, :]
            if sense[i] == "<="
                sense[i] = ">="
            elseif sense[i] == ">="
                sense[i] = "<="
            end
            # "=" remains unchanged.
        end
    end

    # Count extra variables.
    slack_count = 0
    surplus_count = 0
    artificial_count = 0
    for s in sense
        if s == "<="
            slack_count += 1
        elseif s == ">="
            surplus_count += 1
            artificial_count += 1
        elseif s == "="
            artificial_count += 1
        else
            error("Unknown constraint type: $s")
        end
    end

    total_vars = n + slack_count + surplus_count + artificial_count

    # Build the initial tableau.
```

```

tableau = zeros(m+1, total_vars+1)
tableau[1:m, 1:n] = A

# Build variable labels and initialize basis.
original_labels = [ "x$i" for i in 1:n ]
extra_labels = String[]
basis = Int[]
artificial_indices = Int[]
surplus_indices = Int[]

col = n + 1
for i in 1:m
    if sense[i] == "<="
        tableau[i, col] = 1.0 # slack variable
        push!(basis, col)
        push!(extra_labels, "s$i")
        col += 1
    elseif sense[i] == ">="
        tableau[i, col] = -1.0 # surplus variable
        push!(surplus_indices, col)
        push!(extra_labels, "s$i")
        col += 1
        tableau[i, col] = 1.0 # artificial variable
        push!(basis, col)
        push!(artificial_indices, col)
        push!(extra_labels, "a$i")
        col += 1
    elseif sense[i] == "="
        tableau[i, col] = 1.0 # artificial variable
        push!(basis, col)
        push!(artificial_indices, col)
        push!(extra_labels, "a$i")
        col += 1
    end
end

var_labels = vcat(original_labels, extra_labels)
headers = vcat(var_labels, ["RHS"])

# Set the RHS.
tableau[1:m, end] = b

# Big-M value.
M = 1e6

# For mode=:min, flip z so we solve a maximization problem.
c_vec = mode == :min ? -z : z

```

```

# Set up the objective row for maximization.
# For decision variables, store -c_vec.
tableau[end, 1:n] = -c_vec

# For artificial variables, assign a penalty of M.
for j in artificial_indices
    tableau[end, j] = M
end

# Adjust the objective row for rows with an artificial variable.
for i in 1:m
    if basis[i] in artificial_indices
        tableau[end, :] -= M * tableau[i, :]
    end
end

println("Initial Tableau:")
pretty_table(tableau, header=headers)
println("Initial basis indices: ", basis)
println("Artificial variable indices: ", artificial_indices)
println("Surplus variable indices: ", surplus_indices)

iteration = 0
while true
    iteration += 1
    println("\nIteration $iteration:")
    pretty_table(tableau, header=headers)

    # Compute the improvement row: improvement = - (current
    improvement = [ -tableau[end, j] for j in 1:total_vars
    println("Improvement row: ", improvement)

    # Select entering variable (highest positive improvement)
    entering_col = argmax(improvement)
    if improvement[entering_col] <= 0
        println("Optimal solution reached. No positive improvement")
        break
    end
    println("Entering variable: ", headers[entering_col], "

    # Ratio test to choose leaving variable.
    ratios = []
    for i in 1:m
        if tableau[i, entering_col] > 1e-10
            push!(ratios, (tableau[i, end] / tableau[i, entering_col]))
        end
    end
    if isempty(ratios)
        error("The solution is unbounded.")
    end
end

```

```

        error("The solution is unbounded.",
    end

    sorted_ratios = sort(ratios, by = x -> x[1])
    pivot_row = sorted_ratios[1][2]
    pivot_val = tableau[pivot_row, entering_col]

    println("Leaving variable: Row ", pivot_row, " (", head
    println("Pivoting on element at row ", pivot_row, ", co

    # Pivot: normalize the pivot row.
    tableau[pivot_row, :] /= pivot_val
    println("After normalizing pivot row:")
    pretty_table(tableau[pivot_row:pivot_row, :], header=he

    # Update other rows.
    for i in 1:(m+1)
        if i != pivot_row
            factor = tableau[i, entering_col]
            tableau[i, :] -= factor * tableau[pivot_row, :]
            println("Updating row ", i, " (factor: ", facto
            pretty_table(tableau[i:i, :], header=headers)
        end
    end

    # Update basis.
    basis[pivot_row] = entering_col
    println("Updated basis indices: ", basis)

    # === Remove nonbasic artificial variable columns on the
    # Identify all artificial columns that are not in the c
    nonbasic_artificial = [ j for j in artificial_indices i
    if !isempty(nonbasic_artificial)
        println("Removing non-basic artificial variables: "

        # Build a list of columns to keep: all from 1:total
        keep_columns = [ j for j in 1:total_vars if !(j in
        new_total_vars = length(keep_columns)

        # Rebuild the tableau: keep the selected columns and
        tableau = tableau[:, vcat(keep_columns, [total_vars

        # Update variable labels and headers.
        var_labels = var_labels[keep_columns]
        headers = vcat(var_labels, ["RHS"])

        # Re-map basis indices.
        basis = [ findfirst(isequal(old), keep_columns) for

```

```

        # Remap artificial_indices and surplus_indices.
        artificial_indices = [ findfirst(isequal(old), keep.
        surplus_indices = [ findfirst(isequal(old), keep_co
        total_vars = new_total_vars
    end
    # === End removal ===
end

# Extract solution for original decision variables.
solution = zeros(total_vars)
for i in 1:m
    solution[basis[i]] = tableau[i, end]
end

optimum = mode == :min ? -tableau[end, end] : tableau[end,

println("\nFinal Optimal Tableau:")
pretty_table(tableau, header=headers)
println("Optimal solution (", join(original_labels, ", "),
println("Optimal objective value: ", optimum)

return solution[1:n], optimum, tableau, basis
end

```

simplexBigM (generic function with 1 method)

กำหนดตัวแปร: x_1 = Chicken (servings/day) x_2 = Rice x_3 = Broccoli x_4 = Apple x_5 = Milk

Objective (min cost): minimize $25x_1 + 5x_2 + 10x_3 + 8x_4 + 12x_5$

Constraints : Calories ≥ 2000 Protein ≥ 50 Fat ≤ 70 Vegetable+Fruit ≥ 2 servings (Broccoli + Apple ≥ 2) $x \geq 0$

```

# Decision variables: x1=Chicken, x2=Rice, x3=Broccoli, x4=Apple, x
# Minimize cost: 25x1 + 5x2 + 10x3 + 8x4 + 12x5

A = [250.0 200.0 50.0 100.0 150.0; # calories
      30.0 4.0 3.0 0.5 8.0; # protein
      5.0 1.0 0.5 0.2 4.0; # fat
      0.0 0.0 1.0 1.0 0.0] # veg+fruit

b = [2000.0, 50.0, 70.0, 2.0]
z = [25.0, 5.0, 10.0, 8.0, 12.0]
sense = [">=", ">=", "<=", ">="]

```

```
solution, optimum, final_tableau, basis = simplexBigM(A, b, z, sens

println("Minimum cost = ", optimum)
println("x = ", solution)
```

Initial Tableau:

MethodError: no method matching

_text__print_table(::PrettyTables.PrintingSpec;

header::Vector{String})

This error has been manually thrown, explicitly, so the method may exist but be intentionally marked as unimplemented.

Closest candidates are:

_text__print_table(::PrettyTables.PrintingSpec;

alignment_anchor_fallback, alignment_anchor_regex,

apply_alignment_regex_to_summary_rows, auto_wrap,

column_label_width_based_on_first_line_only, display_size,

equal_data_column_widths, fit_table_in_display_horizontally,

fit_table_in_display_vertically, fixed_data_column_widths,

highlighters, line_breaks, minimum_data_column_widths,

maximum_data_column_widths, overwrite_display,

reserved_display_lines, shrinkable_data_column,

shrinkable_column_minimum_width, style, table_format) got

unsupported keyword argument "header"

@ PrettyTables

~/.julia/packages/PrettyTables/MwVkJ/src/backends/text/text_backend.

Stacktrace:

[1] kwerr(::@NamedTuple{header::Vector{String}}, ::Function,
::PrettyTables.PrintingSpec)

@ Base ./error.jl:165

[2] _printing_backend(::Val{:text},
pspec::PrettyTables.PrintingSpec; is_stdout::Bool,
kwargs::@Kwargs{header::Vector{String}})

@ PrettyTables

~/.julia/packages/PrettyTables/MwVkJ/src/main.jl:512

[3] _pretty_table(io::IO, data::Any, backend::Symbol,
compact_printing::Bool, limit_printing::Bool,
show_omitted_cell_summary::Bool, renderer::Symbol, title::String,
subtitle::String, stubhead_label::String,
row_number_column_label::String, row_labels::Nothing,
row_group_labels::Nothing, column_labels::Nothing,
show_column_labels::Bool, summary_rows::Nothing,
summary_row_labels::Nothing, footnotes::Nothing,
source_notes::String, alignment::Symbol,
column_label_alignment::Nothing,
continuation_row_alignment::Nothing, footnote_alignment::Symbol,
row_label_column_alignment::Symbol,
row_group_label_alignment::Symbol,
row_number_column_alignment::Symbol, source_note_alignment::Symbol,
subtitle_alignment::Symbol, title_alignment::Symbol,
cell_alignment::Nothing, formatters::Nothing,

```

maximum_number_of_columns::Int64, maximum_number_of_rows::Int64,
merge_column_label_cells::Symbol, new_line_at_end::Bool,
show_first_column_label_only::Bool, show_row_numbe...
@ PrettyTables
~/.julia/packages/PrettyTables/MwVkJ/src/main.jl:479
[4] _pretty_table
@ ~/.julia/packages/PrettyTables/MwVkJ/src/main.jl:212
[inlined]
[5] #pretty_table#9
@ ~/.julia/packages/PrettyTables/MwVkJ/src/main.jl:94 [inlined]
[6] pretty_table
@ ~/.julia/packages/PrettyTables/MwVkJ/src/main.jl:37 [inlined]
[7] pretty_table(data::Any;
kwargs::@Kwargs{header::Vector{String}})
@ PrettyTables
~/.julia/packages/PrettyTables/MwVkJ/src/main.jl:11
[8] pretty_table
@ ~/.julia/packages/PrettyTables/MwVkJ/src/main.jl:9 [inlined]
[9] simplexBigM(A::Matrix{Float64}, b::Vector{Float64},
z::Vector{Float64}, sense::Vector{String}; mode::Symbol)
@ Main ./In[3]:104
[10] top-level scope
@ In[6]:13

```

